

БУ ВО
«СУРГУТСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ХАНТЫ-МАНСКИЙСКОГО АВТОНОМНОГО ОКРУГА – ЮГРЫ»
ПОЛИТЕХНИЧЕСКИЙ ИНСТИТУТ

Кафедра информатики и вычислительной техники

Курсовая работа
по дисциплине «Разработка web-приложений»
на тему «чат поддержки, база знаний и поиск»

Выполнил: студент группы 607-42

Говор Сергей Владимирович

Проверил: к. т. н., доцент кафедры ИВТ

Егоров Александр Алексеевич

Сургут – 2026

СОДЕРЖАНИЕ

ВВЕДЕНИЕ.....	3
МОДЕЛЬ РАЗВЕРТЫВАНИЯ СИСТЕМЫ	
1.1. Диаграмма развертывания UML.....	4
1.2. Описание развертывания на виртуальном сервере.....	5
ПРОЕКТИРОВАНИЕ КОМПОНЕНТА	
2.1. Назначение и границы компонента.....	7
2.2. Модель данных и диаграмма классов.....	8
2.3. Диаграмма компонентов и API-контракты.....	8
2.4. Бизнес-правила и ограничения доступа.....	10
UI КОМПОНЕНТЫ И РУКОВОДСТВО ПОЛЬЗОВАТЕЛЯ	
3.1. Перечень экранов и состояний.....	11
3.2. Формы, фильтры и валидация.....	11
3.3. Пользовательские сценарии.....	12
3.4. Текстовое описание интерфейсных компонентов.....	12
3.5 Скриншоты компонентов.....	14
ЗАКЛЮЧЕНИЕ.....	16
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ.....	17
ПРИЛОЖЕНИЕ А. PLANTUML-КОД ДИАГРАММ.....	18

Введение

Настоящая курсовая работа посвящена проектированию и разработке сквозного компонента «Чат поддержки, база знаний и поиск». Данный компонент полностью переносит процесс коммуникации пользователей и внутренней службы технической поддержки в единый контур веб-приложения Notary.

Со стороны заявителя компонент решает задачи быстрого поиска ответов на часто задаваемые вопросы (FAQ) в базе знаний посредством полнотекстового поиска по статьям, а при необходимости — создания официального тикета (обращения) в службу поддержки с последующим ведением текстового диалога в реальном времени.

Со стороны оператора внутренней поддержки компонент предоставляет автоматизированное рабочее место — очередь тикетов, распределение обращений по уровням приоритета, ведение диалога с заявителем, а также инструменты создания и редактирования справочных статей для расширения базы знаний авторами контента.

Цель работы — разработать, спроектировать и задокументировать компонент чата поддержки, базы знаний и поиска для веб-портала Notary, включая пользовательские интерфейсы заявителя и оператора, серверную бизнес-логику управления жизненным циклом обращений, подсистему контроля SLA и модуль оповещений.

Для достижения поставленной цели требуется решить следующие задачи:

1. Изучить индивидуальное задание, архитектурные шаблоны проекта и определить границы ответственности компонента.
2. Построить диаграмму развертывания UML проекта и детально описать инфраструктурную схему функционирования подсистемы поддержки.
3. Разработать реляционную модель данных, построить диаграмму классов UML и связать сущности поддержки с общими сущностями пользователей системы.
4. Спроектировать диаграмму компонентов UML с разделением на подсистемы чата и базы знаний, составить спецификацию API-контрактов взаимодействия.
5. Сформулировать строгие бизнес-правила обработки тикетов и разграничения ролевого доступа.
6. Разработать и описать интерфейсные решения (Angular-компоненты), формы ввода, логику валидации, фильтрации и составить подробное руководство пользователя с эксплуатационными сценариями.

Объект разработки — веб-система Notary, предназначенная для автоматизации нотариальной оценки и взаимодействия с гражданами.

Предмет разработки — изолированный компонент поддержки, чата, ведения базы знаний и поиска со встроенным механизмом контроля SLA и нотификаций.

1. МОДЕЛЬ РАЗВЕРТЫВАНИЯ СИСТЕМЫ

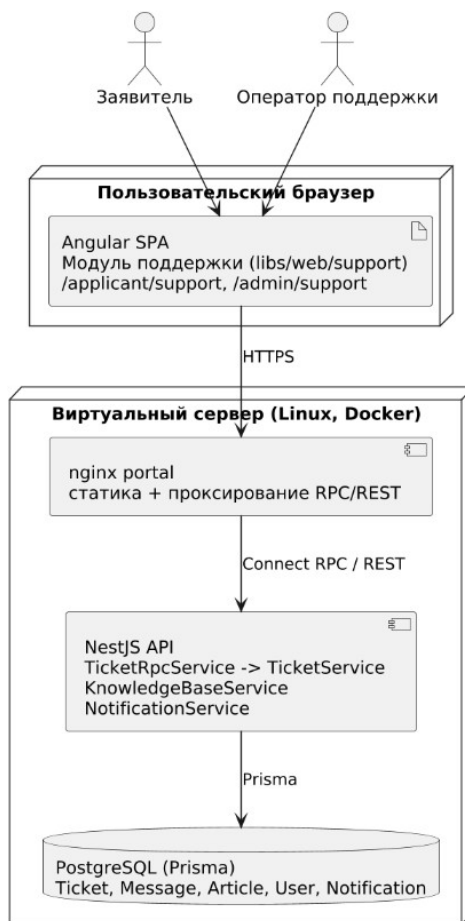
1.1. Диаграмма развертывания UML

Проект разворачивается на виртуальном сервере под управлением операционной системы семейства Linux с использованием технологии контейнеризации Docker и оркестрации Docker Compose. Взаимодействие пользователя с системой осуществляется через современный веб-браузер, в котором выполняется клиентское одностраничное приложение (Angular SPA). Раздачу статических файлов клиентского приложения, а также функции обратного прокси-сервера (reverse proxy) выполняет контейнер nginx portal. Он принимает входящий HTTPS-трафик на портах 80/443 и перенаправляет вызовы RPC-процедур Connect (маршруты /notary.support.*) и REST API (маршруты /api/*) на backend-контейнер NestJS API. Данный подход позволяет клиенту и серверу функционировать в рамках одного источника (Same-Origin policy), избавляя от необходимости дополнительной настройки CORS-политик.

Сервер приложений NestJS осуществляет обработку бизнес-логики, проверку ролей, вычисление SLA и взаимодействует со СУБД PostgreSQL через объектно-реляционное отображение Prisma ORM.

Ниже представлена диаграмма развертывания, на которой акцентировано внимание на контуре прохождения данных компонента поддержки и справки.

Диаграмма развертывания системы: Модуль поддержки и справки



1.2. Описание развертывания на виртуальном сервере

Вся базовая инфраструктура проекта описывается в декларативном файле `docker-compose.yaml`. База данных PostgreSQL выступает единым хранилищем для всех системных модулей. В контексте разработки компонента поддержки, в базу внедрены таблицы для хранения структуры тикетов, истории сообщений в чате, текстового содержимого статей базы знаний и лога системных уведомлений.

Сборка образов приложений автоматизирована с помощью многоэтапных Docker-файлов (`Dockerfile`). Для клиентской части на первом этапе (`Build Stage`) среда Node.js компилирует production-сборку Angular SPA, оптимизируя исходный TypeScript-код, а на втором этапе полученные артефакты переносятся в легковесный образ Nginx, обеспечивающий высокую скорость отдачи статического контента.

Безопасность и изоляция данных на уровне интерфейса гарантируются встроенными механизмами Angular-роутинга (`Route Guards`), которые проверяют авторизационный токен пользователя и его принадлежность к требуемой роли. Заявителям полностью заблокирован доступ к административной панели и очереди тикетов операторов, а операторы не имеют прав на создание тикетов от чужого имени.

Все критически важные параметры конфигурации вынесены из исходного кода в переменные окружения, считываемые сервером на этапе инициализации:

- `DATABASE_URL` — строка подключения к PostgreSQL для Prisma.
- `JWT_SECRET` — криптографический ключ для верификации подписей токенов сессий.
- `SLA_DEFAULT_REPLY_MINUTES` — регламентное время на первичный ответ оператора (по умолчанию 120 минут).

Таблица 1 – Компоненты инфраструктуры и их связь с компонентом поддержки

Элемент	Назначение	Связь с компонентом поддержки
Браузер, Angular SPA	Пользовательский интерфейс портала	Экран <code>/admin/support</code> (только Admin: <code>authGuard</code> , <code>roleGuard</code>); вызовы <code>SupportService</code> по <code>Connect RPC</code>
nginx portal	Контейнер portal: отдаёт статику SPA, проксирует <code>/api/*</code> и <code>Connect RPC (/notary.*)</code> на <code>api:3000</code> ; порт 80	Передаёт вызовы <code>SupportService</code> в API; SPA и API работают в одном <code>origin</code>

Элемент	Назначение	Связь с компонентом поддержки
NestJS API	Контейнер api, порт 3000: бизнес-логика и Connect RPC; проверка JWT в AuthInterceptor	SupportModule: SupportRpcService → SupportService; 9 RPC, права доступа, расчёт SLA
PostgreSQL + Prisma	Реляционное хранилище данных всех модулей	Таблицы tickets, ticket_messages, users (модели Ticket, TicketMessage, User)
Мониторинг: Prometheus, Grafana, Loki, Promtail, Tempo	Метрики, логи и трассировки контейнеров и API	Общие HTTP-логи и метрики API охватывают и вызовы SupportService; отдельных метрик поддержки нет
MinIO: вложения поддержки (проектируется)	S3-хранилище файлов; бакеты payment-documents, assessment-files	Хранение вложений к сообщениям не реализовано: в БД только attachmentIds
GigaChat (проектируется)	Внешний сервис ответов ИИ-ассистента	AskSupportAi возвращает шаблонный ответ; внешнего вызова нет

2. ПРОЕКТИРОВАНИЕ КОМПОНЕНТА

2.1. Назначение и границы компонента

Разрабатываемый компонент поддержки и базы знаний является изолированной подсистемой, предназначенной для агрегации пользовательских обращений и предоставления справочной информации. Взаимодействие с компонентом строго разграничено в зависимости от роли участника:

1. Заявитель (Applicant):

- o Осуществляет поиск по ключевым словам среди опубликованных статей базы знаний.
- o Просматривает структурированный FAQ по категориям.
- o Создает новые тикеты с указанием темы, подробного описания проблемы и степени критичности инцидента.
- o Ведет диалог в чате с оператором внутри конкретного тикета, отслеживая смену его статусов.

2. Оператор поддержки (Support Operator):

- o Работает со специализированным интерфейсом — очередью входящих обращений (Ticket Queue), ранжированных по приоритету и времени нарушения SLA.
- o Берет тикеты в работу, осуществляет консультации в чате и закрывает решенные вопросы.

3. Автор контента / Редактор (Support Author):

- o Осуществляет администрирование базы знаний: создает новые статьи, редактирует существующие материалы, распределяет их по тематическим категориям.

Внешние системы (такие как модуль нотариальной оценки недвижимости, подсистема биллинга и генерации копий документов) находятся вне границ проектируемого компонента и взаимодействуют с ним исключительно на уровне общих идентификаторов пользователей (userId).

Таблица 2 – Основные характеристики компонента поддержки

Элемент	Описание
Роль пользователя	Admin — оператор поддержки, экран /admin/support. Applicant и Notary работают со своими тикетами через API; их экраны /applicant/support и /notary/support (проектируется)
Основной сценарий	Оператор выбирает тикет в очереди, нажимает «Взять» (Open → InProgress), отвечает в чате, отмечает «Решено» (Resolved) или закрывает тикет (Closed) с итоговым решением. Вопрос ИИ-ассистенту и эскалация на оператора — есть в API, экран (проектируется)
Входные данные	Форма тикета: тема, email пользователя, приоритет SLA, описание; текст ответа; фильтры очереди (статус, SLA, приоритет) и сортировка; итоговое решение при закрытии. Поисковый запрос по статьям (проектируется)
Выходные данные	Записи в tickets и ticket_messages; срок SLA (slaDeadline); статус тикета; счётчики «Открытых», «Нарушено SLA», «Решено сегодня». Список найденных статей (проектируется)
Соседние модули	Авторизация: AuthInterceptor, requireAuth, requireRole, requireSelfOrRole; пользователи: User, поиск по email при создании тикета; Prisma. Вложения в MinIO (проектируется); уведомления не подключены
Ограничения	Все RPC — только с токеном; не-Admin видит только свои тикеты; менять статус и закрывать тикет может только Admin; в закрытый тикет нельзя писать и менять ему статус; методов правки и удаления сообщений нет

2.2. Модель данных и диаграмма классов

Для обеспечения целостности и нормализации данных спроектирована схема реляционных таблиц. Центральной сущностью подсистемы общения является класс Ticket (обращение), который связан отношением «многие к одному» с сущностью пользователя User (как в роли создателя-клиента customerId, так и в роли исполнителя assignedToId). Каждое обращение содержит лог текстовых сообщений Message.

Подсистема справки базируется на сущности Article (статья базы знаний), жестко привязанной к её создателю (authorId). При отправке важных ответов или смене ответственных лиц система порождает записи в таблице Notification.

Ниже представлена концептуальная диаграмма классов UML, содержащая типы данных, первичные и внешние ключи, а также кратности связей.

Диаграмма классов: чат поддержки, база знаний и поиск

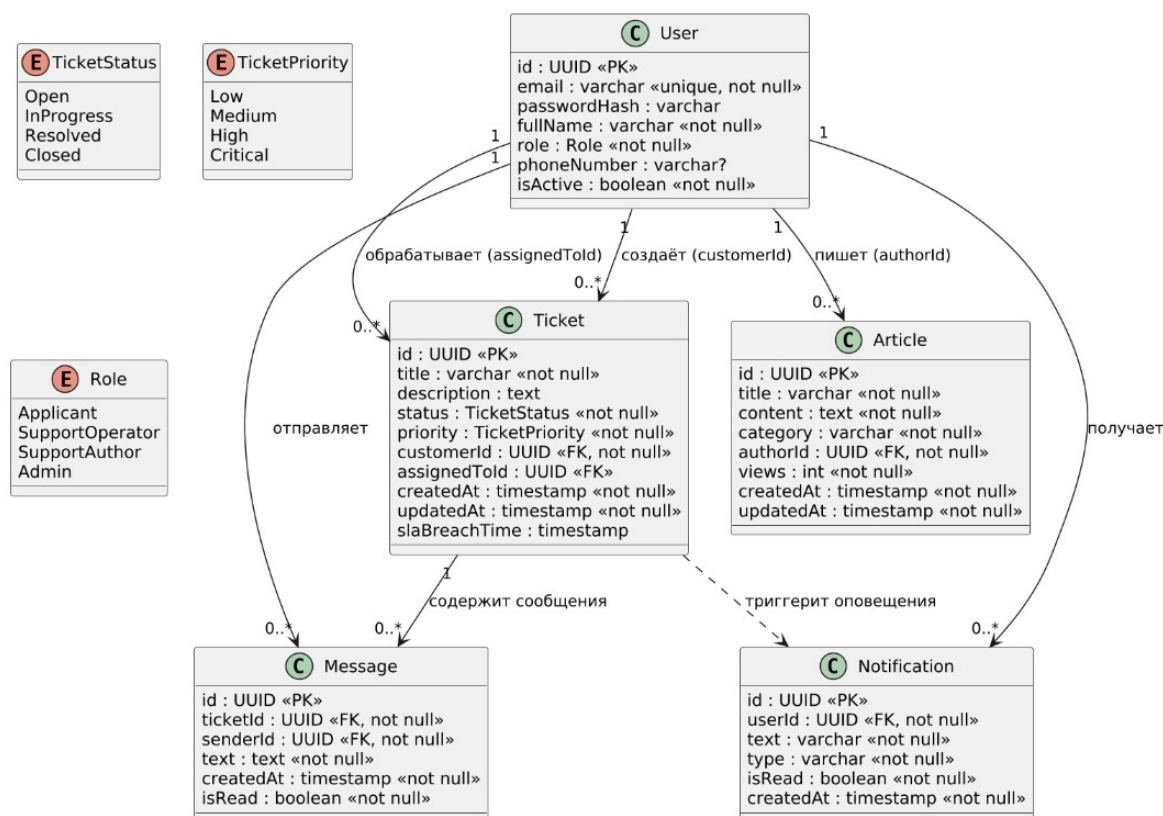


Таблица 3 – Сущности компонента

Сущность	Назначение в компоненте	Ключевые поля и связи
Ticket (tickets)	Обращение в поддержку	id (UUID, PK); subject; status: TicketStatus — Open, InProgress, Resolved, Closed (по умолчанию Open); priority: TicketPriority — Low, Medium, High, Urgent (по умолчанию Medium); authorId → User; assigneeId → User, необязательный; slaDeadline; resolution; resolvedAt; createdAt; updatedAt; messages: TicketMessage[] (1:N)
TicketMessage (ticket_messages)	Сообщение в переписке по тикету	id (UUID, PK); ticketId → Ticket, каскадное удаление; authorId → User; role: TicketMessageRole — User, Ai, Support (по умолчанию User); text; attachmentIds: String[] — идентификаторы вложений; createdAt

Сущность	Назначение в компоненте	Ключевые поля и связи
User (users)	Автор тикетов и сообщений; Admin — исполнитель	id (UUID, PK); email (unique); fullName; role: Role — Applicant, Notary, Admin (по умолчанию Applicant); supportTickets, assignedTickets: Ticket[] (1:N); ticketMessages: TicketMessage[] (1:N)
Article (проектируется)	Статья базы знаний для FAQ и поиска	Концептуальная сущность, в schema.prisma отсутствует; планируемые поля: id, title, content, category, authorId → User

2.3. Диаграмма компонентов и API-контракты

В соответствии с требованиями по четкому разделению ответственности подсистем, архитектура компонента разделена на два крупных домена: Подсистема тикетов и чата (отвечает за оперативную поддержку) и Подсистема базы знаний (отвечает за статический контент и поиск). Взаимодействие между Angular-модулями и NestJS-контроллерами специфицировано посредством строгих контрактов протокола Connect RPC (Protobuf).

Диаграмма компонентов: Чат поддержки и База знаний

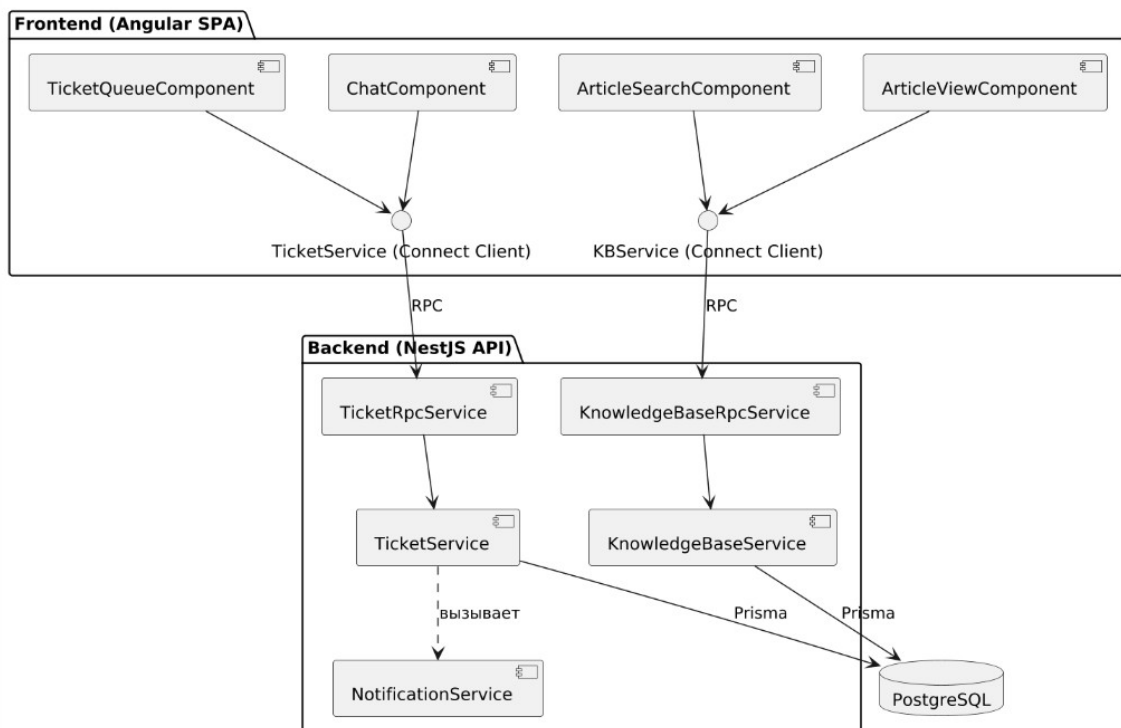


Таблица 4 – API-методы компонента

RPC	Назначение	Доступ	Параметры → ответ	Ошибки	Статус
SupportService. AskSupportAi	Ответ ИИ-ассистента на вопрос	Все роли	question_text → answer_text	Unauthenticated; InvalidArgument (пустой вопрос)	Реализован (ответ-заглушка)
SupportService. EscalateToOperator	Тикет из диалога с ИИ: приоритет Medium, история — в сообщении	Все роли	subject, history[] (role, text) → ticket	Unauthenticated; InvalidArgument (пустой subject)	Реализован
SupportService. CreateTicket	Создать тикет; text — первое сообщение	Все роли; от имени другого только Admin	subject, text, priority, author_email (опц.) → ticket	Unauthenticated; InvalidArgument (пустой subject); PermissionDenied; NotFound (нет пользователя с email)	Реализован
SupportService. GetTicket	Карточка тикета	Автор или Admin	id (UUID) → ticket	Unauthenticated; InvalidArgument (не UUID); NotFound; PermissionDenied	Реализован
SupportService. ListTickets	Список тикетов, новые сверху	Admin: все; другие: свои	pagination (page, limit ≤ 200), status_filter → tickets[], meta	Unauthenticated	Реализован
SupportService. AddMessage	Сообщение в тикет; ответ Admin назначает его исполнителем, Open/Resolved → InProgress	Автор или Admin	ticket_id, text, attachment_ids → message, ticket	Unauthenticated; InvalidArgument; NotFound; PermissionDenied; FailedPrecondition (тикет закрыт)	Реализован
SupportService. ListMessages	История сообщений в порядке создания	Автор или Admin	ticket_id, pagination → messages[], meta	Unauthenticated; InvalidArgument; NotFound; PermissionDenied	Реализован
SupportService. UpdateTicketStatus	Сменить статус; InProgress назначает исполнителя, Resolved и Closed ставят resolvedAt	Только Admin	id, status → ticket	Unauthenticated; PermissionDenied; InvalidArgument; NotFound; FailedPrecondition (тикет закрыт)	Реализован
SupportService. CloseTicket	Закрыть тикет с итоговым решением	Только Admin	id, resolution → ticket	Unauthenticated; PermissionDenied; InvalidArgument; NotFound; FailedPrecondition (уже закрыт)	Реализован
KBService. SearchArticles	Поиск статей базы знаний	Все роли	query, category → articles[]	Unauthenticated	Проектируется
KBService. CreateArticle	Создание статьи базы знаний	Только Admin	title, content, category → article	Unauthenticated; PermissionDenied; InvalidArgument	Проектируется

Примечание: контракт —

libs/shared/api-contracts/proto/notary/support/v1alpha1/support.proto;

обработчики — `libs/api/support/src/lib/support-rpc.service.ts` и `support.service.ts`; регистрация — `apps/api/src/app/connect-router.registry.ts`. `Unauthenticated` при отсутствии токена возвращает `AuthInterceptor`; `KBService` — проектируемый контракт.

2.4. Бизнес-правила и ограничения доступа

Функционирование компонента подчинено строгим логическим условиям и правилам разграничения ролей:

Правила SLA (Service Level Agreement): Каждому тикету при создании, в зависимости от выбранного приоритета (`TicketPriority`), рассчитывается точное предельное время ответа (`slaBreachTime`). Для приоритета `Critical` время ответа составляет 30 минут, для `High` — 2 часа, для `Medium` — 4 часа, для `Low` — 24 часа. Если оператор не оставляет сообщение до наступления этой метки, тикет помечается в системе как «SLA Violated» и поднимается в самый верх очереди административной панели.

Жизненный цикл тикета: Обращение может переходить по состояниям строго последовательно: `Open` (Новый) -> `InProgress` (В работе) -> `Resolved` (Решен) -> `Closed` (Закрыт). Заявитель имеет право перевести свой тикет в статус `Closed` на любом этапе, если проблема отпала. Оператор переводит тикет в `InProgress` в момент нажатия кнопки «Взять в работу».

Ограничения чата: Удаление или редактирование сообщений после отправки запрещено для обеспечения истории проведения аудита инцидентов.

Таблица 5 – Бизнес-правила и ограничения доступа

Правило	Описание
Доступ к экрану и API	<code>/admin/support</code> открывается только роли <code>Admin</code> (<code>authGuard</code> , <code>roleGuard</code>); все RPC <code>SupportService</code> требуют <code>Bearer</code> -токен, иначе <code>Unauthenticated</code>
Видимость тикетов	<code>Admin</code> видит все тикеты, остальные роли — только свои (<code>authorId</code>); <code>GetTicket</code> , <code>ListMessages</code> и <code>AddMessage</code> доступны автору или <code>Admin</code> , иначе <code>PermissionDenied</code>
Создание тикета	Любая роль создаёт тикет от своего имени; от имени другого пользователя (<code>author_email</code>) — только <code>Admin</code> . Новый тикет получает статус <code>Open</code> и срок SLA
Жизненный цикл и смена статуса	Статусы <code>Open</code> , <code>InProgress</code> , <code>Resolved</code> , <code>Closed</code> . Менять статус (<code>UpdateTicketStatus</code>) и закрывать тикет (<code>CloseTicket</code>) может только <code>Admin</code> ; порядок переходов не проверяется, запрещено лишь менять закрытый тикет. <code>InProgress</code> назначает <code>Admin</code> исполнителем; <code>Resolved</code> и <code>Closed</code> записывают <code>resolvedAt</code>

Правило	Описание
SLA	slaDeadline = время создания + 1 ч (Urgent), 4 ч (High), 24 ч (Medium), 72 ч (Low). Статус SLA вычисляет интерфейс: «Нарушен» — срок прошёл, «Скоро нарушится» — осталось меньше 2 ч, иначе «В норме»
Сообщения и роли сообщений	Писать (AddMessage) могут автор тикета и Admin: роль Support — у Admin, User — у остальных. Первое сообщение CreateTicket всегда User; EscalateToOperator берёт роли (User, Ai, Support) из присланной истории. Ответ Admin назначает его исполнителем и переводит Open/Resolved → InProgress. Методов правки и удаления сообщений нет
Закрытый тикет	В тикет со статусом Closed нельзя добавить сообщение и сменить ему статус — FailedPrecondition; в интерфейсе поле ответа скрыто
Подтверждение действий	«Решено» и «Закрыть» требуют подтверждения (confirm); при закрытии запрашивается итоговое решение, необязательное
ИИ-ассистент и эскалация на оператора	AskSupportAi возвращает шаблонный ответ (заглушка GigaChat); EscalateToOperator создаёт тикет с приоритетом Medium и переносит непустые сообщения истории. В интерфейсе не используются
Ошибки для пользователя	Ошибки загрузки и создания тикета выводятся текстом над очередью; при смене статуса и отправке сообщения ошибки не обрабатываются
Логирование	Отдельного аудита действий с тикетами нет; запросы попадают в общий HTTP-лог API
База знаний и поиск (проектируется)	Статей и поиска в коде нет; /faq, /applicant/faq и /notary/faq — заглушки. Чтение статей — всем ролям, ведение — Admin

3. UI КОМПОНЕНТЫ И РУКОВОДСТВО ПОЛЬЗОВАТЕЛЯ

3.1. Перечень экранов и состояний

Пользовательский интерфейс модуля интегрирован в общую дизайн-систему портала Notary. Разработаны следующие основные экраны:

1. Экран Базы знаний (/applicant/support/faq): Стартовая страница поддержки для заявителя. Содержит крупную поисковую строку по центру и сетку категорий (например, «Оплата услуг», «Документы для оценки», «Технические сбои»).
2. Экран списка обращений (/applicant/support/tickets): Таблица личных тикетов заявителя с отображением текущего статуса, даты обновления и темы.
3. Рабочее место оператора (/admin/support/queue): Сводная таблица всех открытых тикетов в системе. Поддерживает динамическое обновление данных без перезагрузки страницы (веб-сокеты / лонг-пуллинг).
4. Экран чата по тикету (/support/tickets/:id): Двухпанельный интерфейс: слева — метаданные тикета, данные контрагента и таймер SLA; справа — окно диалога с прокруткой и полем ввода.

Таблица 6 – Перечень экранов компонента

Экран	Маршрут	Роль	Основные элементы	Действия
Страница поддержки (lib-admin-support)	/admin/support	Admin	Счётчики «Открытых», «Нарушено SLA», «Решено сегодня»; фильтры статуса, SLA и приоритета; сортировка; «Загрузка тикетов...»; текст ошибки	Фильтровать и сортировать; «+ Создать тикет»; список обновляется каждые 10 с
Очередь тикетов (lib-ticket-queue)	/admin/support	Admin	Колонки: тема и email, дата создания, статус, SLA, приоритет, осталось времени, действия; пустой список — «Нет тикетов, соответствующих фильтрам»	Выбрать тикет; «Взять» у тикета Open → InProgress

Экран	Маршрут	Роль	Основные элементы	Действия
Чат оператора (lib-operator-chat)	/admin/support	Admin	Автор: email, дата регистрации, метка роли (user/admin — вычисляется по email); история сообщений; поле ответа, скрытое у Closed; «Выберите тикет из списка для начала работы»	Отправить ответ; «Решено»; «Заккрыть» с итоговым решением
Окно создания тикета (lib-create-ticket-modal)	/admin/support	Admin	Тема, email пользователя, роль пользователя, приоритет SLA, описание; «Отмена», «Создать тикет»	Создать тикет от имени пользователя (CreateTicket с author_email)
Чат поддержки (проектируется)	/applicant/support, /notary/support	Applicant, Notary	Заглушка placeholder: «Чат/тикеты», «Вложения»; у заявителя также «SLA-статусы»	—
Справочник: база знаний, FAQ, поиск (проектируется)	/applicant/faq, /notary/faq	Applicant, Notary	Заглушка placeholder: «База знаний», «FAQ», «Поиск по статьям»	—

3.2. Формы, фильтры и валидация

Форма создания тикета содержит валидацию на стороне клиента (Angular Reactive Forms). Поля «Тема» (минимум 10 символов) и «Описание проблемы» являются строго обязательными. Кнопка отправки формы остается заблокированной (disabled), пока все условия валидации не выполнены.

На экране очереди оператора реализован блок фильтрации. Доступна сортировка по уровню критичности, по статусу, а также фильтр «Только мои тикеты». Нарушенные по времени SLA тикеты подсвечиваются мягким красным фоном строки.

Таблица 7 – Поля формы создания тикета и правила проверки

Поле	Тип	Обязательное	Проверка	Сообщение об ошибке
Тема тикета (subject)	input text	Да	UI: не пусто после trim, иначе кнопка «Создать тикет» неактивна. Сервер: пусто после trim → InvalidArgument; min_len = 1 из proto отдельно не проверяется (protovalidate не подключён)	— (кнопка неактивна)
Email пользователя (author_email)	input email	Да	UI: не пусто и формат /^[^s@]+@[^s@]+\.[^s@]+\$/ — иначе кнопка неактивна. Сервер: author_email принимается только от Admin; пользователь с таким email должен существовать	— в форме; ошибка сервера «[not_found] User with email ... not found» выводится над очередью
Роль пользователя	select: user, vip, admin	Нет	Нет; значение на сервер не передаётся	—

Поле	Тип	Обязательное	Проверка	Сообщение об ошибке
Приоритет SLA (priority)	select: critical, high, medium, low	Да, по умолчанию medium	Значение всегда выбрано; critical → Urgent. Сервер: не задан → Medium	—
Описание проблемы (text)	textarea	Нет	Нет; непустое после trim сохраняется первым сообщением с ролью User	—

3.3. Пользовательские сценарии

1. Поиск информации и создание тикета (Заявитель):

- Заявитель пытается решить возникшую проблему самостоятельно, используя строку полнотекстового поиска на экране FAQ. Клиентское приложение отправляет RPC-запрос SearchArticles.
- Если среди возвращенных записей СУБД подходящего решения не нашлось, пользователь открывает форму обратной связи, заполняет обязательные поля и выбирает критичность. Сервис TicketRpcService регистрирует в базе данных сущность Ticket со статусом Open и автоматически рассчитывает метку времени slaBreachTime (лимит времени на реакцию персонала). Интерфейс выполняет роутинг в созданную карточку чата.

2. Обработка тикета из очереди (Оператор поддержки):

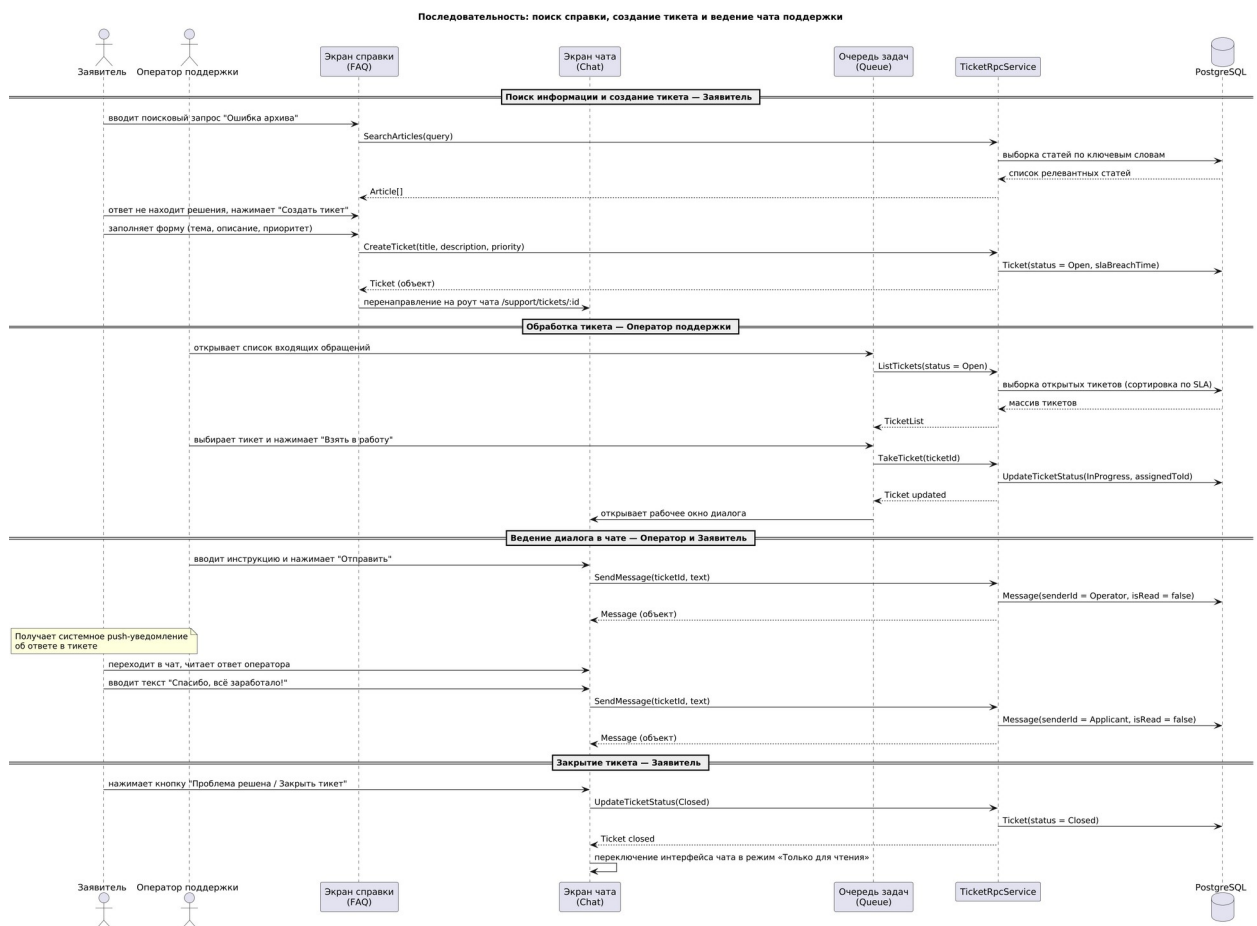
- Оператор службы поддержки обновляет экран общей очереди задач (/admin/support/queue). Компонент запрашивает список активных инцидентов через метод ListTickets. СУБД возвращает записи, отсортированные по уровню приоритета и близости нарушения регламента SLA.
- Оператор кликает по карточке инцидента и берет его на исполнение (TakeTicket). На уровне базы данных фиксируется идентификатор исполнителя (assignedToId), а статус тикета переходит в значение InProgress.

3. Ведение диалога в чате (Оператор и Заявитель):

- Оператор пишет текстовую рекомендацию по устранению сбоя и отправляет её в чат. Вызывается RPC-метод SendMessage, создающий запись в таблице Message.
- В этот момент фоновый модуль бэкенда NotificationService генерирует моментальное push-уведомление для заявителя. Заявитель заходит в личный кабинет, видит сообщение от поддержки, выполняет указанные действия и отправляет подтверждающий ответ обратно в чат.

4. Закрытие тикета (Заявитель):

- Убедившись, что технический сбой устранен, заявитель нажимает на интерфейсную кнопку закрытия диалога. Компонент чата отправляет финальную команду обновления статуса.
- В базе данных статус сущности Ticket окончательно меняется на Closed. На клиенте форма ввода текста блокируется, а вся история переписки фиксируется в неизменяемом виде для последующего контроля качества работы службы поддержки.



3.4. Скриншоты компонента

Нотариальная оценка

Главное меню

Пользователи

Управление заказами

Управление статусами

Очередь оценок

Ручная модерация

История статусов заявок

Платежи

Подписки

Тарифные планы

Скидки

Промокоды

Панель администратора

ОТКРЫТЫХ1

НАРУШЕНО SLA2

РЕШЕНО СЕГОДНЯ0

Фильтрация

+ Создать тикет

Статус

Все статусы

SLA Статус

Любой SLA

Приоритет SLA

Любой приоритет

Сортировка

Сначала новые

Тема / Пользователь	Дата создания	Статус	SLA	Приоритет	Осталось времени	Действия
Ошибка при рендере графиков seed-user-001@seed.local	18.06.2026 17:33	IN_PROGRESS	Нарушен	Высокий	Просрочено	
Не проходит оплата картой МИР seed-user-000@seed.local	18.06.2026 15:03	OPEN	Нарушен	Критический	Просрочено	Взять

Выберите тикет из списка для начала работы

Основная панель

ОТКРЫТЫХ1

НАРУШЕНО SLA1

РЕШЕНО СЕГОДНЯ0

Фильтрация

+ Создать тикет

Статус

Все статусы

SLA Статус

Любой SLA

Приоритет SLA

Любой приоритет

Сортировка

Сначала новые

Тема / Пользователь	Дата создания	Статус	SLA	Приоритет	Осталось времени	Действия
Ошибка при рендере графиков seed-user-001@seed.local	20.06.2026 00:39	IN_PROGRESS	Скоро нарушится	Высокий	1 ч 29 мин	
Не проходит оплата картой МИР seed-user-000@seed.local	19.06.2026 22:09	OPEN	Нарушен	Критический	Просрочено	Взять

Информация о пользователе

Email: seed-user-001@seed.local Роль: USER

Регистрация: 18.06.2026

Решено Закрывать

Заявитель 2 (user)
В личном кабинете не отображаются графики за прошлую неделю.
00:39

Введите ответ от имени поддержки...

Отправить

Основная панель с окном чата

18

Создание нового тикета

×

Тема тикета *

Кратко опишите проблему

Email пользователя *

user@example.com

Роль пользователя

Обычный пользователь

Приоритет SLA *

Средний (ответ в течение 24 часов)

Описание проблемы

Подробно опишите проблему...

Отмена

Создать тикет

Окно создания тикета

Заключение

В ходе выполнения курсовой работы был полностью спроектирован, структурирован и детально описан изолированный функциональный компонент «Чат поддержки, база знаний и поиск» для веб-портала автоматизации нотариальной деятельности Notary.

В рамках архитектурного проектирования были успешно решены все поставленные задачи:

1. Детализирована модель развертывания программного комплекса в контейнеризированной среде Linux/Docker с распределением портов, проксированием трафика через nginx и изоляцией ролей.
2. Создана нормализованная реляционная база данных и построена комплексная диаграмма классов UML, объединившая сущности тикетов, мгновенных сообщений, структурированных статей FAQ и системных пуш-нотификаций.
3. Разработана диаграмма компонентов с разделением общей архитектурной зоны ответственности на две независимые подсистемы (чат коммуникации и поисковый модуль справки), взаимодействующие по протоколу Connec RPC через четко документированные контракты.

4. Сформулированы жесткие бизнес-правила автоматического контроля времени ответа сотрудников (SLA) и переходов по жизненному циклу обращений.
5. Подготовлено развернутое руководство пользователя с описанием трех базовых эксплуатационных сценариев для заявителей и операторов, а также спроектирована клиентская валидация экранных форм.

Разработанный компонент позволяет исключить использование сторонних систем связи, повышает скорость обработки технических инцидентов, обеспечивает прозрачный контроль качества работы операторов и предоставляет пользователям удобный механизм самопомощи через базу знаний.

Список используемых источников

1. Архитектурный шаблон и структура курсового задания проекта Notary: docs/coursework/structure.md.
2. Матрица распределения зон ответственности участников: docs/Responsobility.md.
3. Документация NestJS — Проектирование микросервисов и RPC-сервисов: [Электронный ресурс]. URL: <https://docs.nestjs.com/>
4. Руководство по разработке интерфейсов Angular (Reactive Forms & Routing): [Электронный ресурс]. URL: <https://angular.dev/>
5. Спецификация протокола Connect RPC для TypeScript и Node.js: [Электронный ресурс]. URL: <https://connectrpc.com/>

Приложение а. plantuml-код диаграмм

В данном приложении содержится исходный PlantUML-код для генерации всех графических схем и диаграмм, включенных в курсовую работу.

Исходный код диаграммы развертывания (Рисунок 1)

```
@startuml
```

```
title Диаграмма развертывания системы: Модуль поддержки и справки
```

```
skinparam defaultFontName DejaVu Sans
```

```
skinparam dpi 150
```

```
skinparam shadowing false
```

```
actor "Заявитель" as appl
```

```
actor "Оператор поддержки" as support
```

```
node "Пользовательский браузер" as browser {
```

```
    artifact "Angular SPA\nМодуль поддержки  
(libs/web/support)\n/applicant/support, /admin/support" as spa  
}
```

```

node "Виртуальный сервер (Linux, Docker)" as server {
    component "nginx portal\nстатика + проксирование RPC/REST" as nginx
    component "NestJS API\nTicketRpcService -> TicketService\nKnowledgeBaseService\nNotificationService" as api
    database "PostgreSQL (Prisma)\nTicket, Message, Article, User, Notification"
as pg
}

```

```

appl --> spa
support --> spa
spa --> nginx : HTTPS
nginx --> api : Connect RPC / REST
api --> pg : Prisma
@enduml

```

Исходный код диаграммы классов (Рисунок 2)

Фрагмент кода

```

@startuml
title Диаграмма классов: чат поддержки, база знаний и поиск
skinparam defaultFontName DejaVu Sans
skinparam dpi 150
skinparam shadowing false
skinparam classAttributeIconSize 0

```

```

enum TicketStatus {
    Open
    InProgress
    Resolved
    Closed
}

```

```

enum TicketPriority {
    Low
    Medium
    High
    Critical
}

```

```

enum Role {
    Applicant
    SupportOperator
    SupportAuthor
    Admin
}

```

```

class User {
    id : UUID «PK»
    email : varchar «unique, not null»
    passwordHash : varchar
    fullName : varchar «not null»
    role : Role «not null»
    phoneNumber : varchar?
    isActive : boolean «not null»
}

class Ticket {
    id : UUID «PK»
    title : varchar «not null»
    description : text
    status : TicketStatus «not null»
    priority : TicketPriority «not null»
    customerId : UUID «FK, not null»
    assignedToId : UUID «FK»
    createdAt : timestamp «not null»
    updatedAt : timestamp «not null»
    slaBreachTime : timestamp
}

class Message {
    id : UUID «PK»
    ticketId : UUID «FK, not null»
    senderId : UUID «FK, not null»
    text : text «not null»
    createdAt : timestamp «not null»
    isRead : boolean «not null»
}

class Article {
    id : UUID «PK»
    title : varchar «not null»
    content : text «not null»
    category : varchar «not null»
    authorId : UUID «FK, not null»
    views : int «not null»
    createdAt : timestamp «not null»
    updatedAt : timestamp «not null»
}

```

```
class Notification {
  id : UUID «PK»
  userId : UUID «FK, not null»
  text : varchar «not null»
  type : varchar «not null»
  isRead : boolean «not null»
  createdAt : timestamp «not null»
}
```

```
User "1" --> "0..*" Ticket : "создаёт (customerId)"
User "1" --> "0..*" Ticket : "обрабатывает (assignedToId)"
User "1" --> "0..*" Message : "отправляет"
User "1" --> "0..*" Article : "пишет (authorId)"
User "1" --> "0..*" Notification : "получает"
```

```
Ticket "1" --> "0..*" Message : "содержит сообщения"
Ticket ..> Notification : "триггерит оповещения"
@enduml
```

Исходный код диаграммы компонентов (Рисунок 3)
Фрагмент кода

@startuml

```
title Диаграмма компонентов: Чат поддержки и База знаний
skinparam defaultFontName DejaVu Sans
skinparam dpi 150
skinparam shadowing false
```

```
package "Frontend (Angular SPA)" {
  component "TicketQueueComponent" as queue
  component "ChatComponent" as chatUI
  component "ArticleSearchComponent" as searchUI
  component "ArticleViewComponent" as viewUI
  interface "TicketService (Connect Client)" as ticketClient
  interface "KBService (Connect Client)" as kbClient
}
```

```
package "Backend (NestJS API)" {
  component "TicketRpcService" as ticketRpc
  component "KnowledgeBaseRpcService" as kbRpc
  component "TicketService" as ticketCore
  component "KnowledgeBaseService" as kbCore
  component "NotificationService" as notifyCore
}
```

database "PostgreSQL" as db

queue --> ticketClient

chatUI --> ticketClient

searchUI --> kbClient

viewUI --> kbClient

ticketClient --> ticketRpc : RPC

kbClient --> kbRpc : RPC

ticketRpc --> ticketCore

kbRpc --> kbCore

ticketCore --> db : Prisma

kbCore --> db : Prisma

ticketCore ..> notifyCore : вызывает

@enduml

Исходный код диаграммы сценариев

@startuml

title Последовательность: поиск справки, создание тикета и ведение чата поддержки

skinparam defaultFontName DejaVu Sans

skinparam dpi 150

skinparam shadowing false

actor "Заявитель" as A

actor "Оператор поддержки" as O

participant "Экран справки\n(FAQ)" as FAQ

participant "Экран чата\n(Chat)" as CHAT

participant "Очередь задач\n(Queue)" as Q

participant "TicketRpcService" as S

database "PostgreSQL" as DB

== Поиск информации и создание тикета — Заявитель ==

A -> FAQ : вводит поисковый запрос "Ошибка архива"

FAQ -> S : SearchArticles(query)

S -> DB : выборка статей по ключевым словам

DB --> S : список релевантных статей

S --> FAQ : Article[]

A -> FAQ : ответ не находит решения, нажимает "Создать тикет"

A -> FAQ : заполняет форму (тема, описание, приоритет)

FAQ -> S : CreateTicket(title, description, priority)

S -> DB : Ticket(status = Open, slaBreachTime)

S --> FAQ : Ticket (объект)

FAQ -> CHAT : перенаправление на роут чата /support/tickets/:id

== Обработка тикета — Оператор поддержки ==

O -> Q : открывает список входящих обращений

Q -> S : ListTickets(status = Open)

S -> DB : выборка открытых тикетов (сортировка по SLA)

DB --> S : массив тикетов

S --> Q : TicketList

O -> Q : выбирает тикет и нажимает "Взять в работу"

Q -> S : TakeTicket(ticketId)

S -> DB : UpdateTicketStatus(InProgress, assignedToId)

S --> Q : Ticket updated

Q -> CHAT : открывает рабочее окно диалога

== Ведение диалога в чате — Оператор и Заявитель ==

O -> CHAT : вводит инструкцию и нажимает "Отправить"

CHAT -> S : SendMessage(ticketId, text)

S -> DB : Message(senderId = Operator, isRead = false)

S --> CHAT : Message (объект)

Note over A : Получает системное push-уведомление\поб ответе в тикете

A -> CHAT : переходит в чат, читает ответ оператора

A -> CHAT : вводит текст "Спасибо, всё заработало!"

CHAT -> S : SendMessage(ticketId, text)

S -> DB : Message(senderId = Applicant, isRead = false)

S --> CHAT : Message (объект)

== Закрытие тикета — Заявитель ==

A -> CHAT : нажимает кнопку "Проблема решена / Закрыть тикет"

CHAT -> S : UpdateTicketStatus(Closed)

S -> DB : Ticket(status = Closed)

S --> CHAT : Ticket closed

CHAT -> CHAT : переключение интерфейса чата в режим «Только для чтения»

@startuml